

# OT/ICS Security Portfolio — Project Case Studies

A technical summary for non-technical readers

Prepared by: Carmelino A. Neto | Full technical documentation, code, and evidence: [github.com/carmelin-neto](https://github.com/carmelin-neto)

## Why This Document Exists

Industrial facilities — oil & gas, manufacturing, utilities — run two kinds of computer networks side by side: ordinary office IT, and the operational technology (OT) that directly controls physical equipment like valves, pumps, and sensors. A cyberattack on office IT is a data problem. A cyberattack on OT is a safety problem — it can mean a physical machine doing something it shouldn't.

The nine projects below follow one continuous body of work: designing a defensible network, testing it against real attacks, building the tools to detect those attacks, scoring the resulting risk formally, responding to an incident when one occurs, restricting who can do what once inside, verifying the whole environment for weaknesses a scanner would find, protecting a web-facing application, and putting AI-assisted tooling to work on real detection data — with its output checked, not just trusted. Each project includes a real, working GitHub repository with screenshots, code, and step-by-step evidence.

---

## Project 1: Designing and Proving a Segmented Network

### Business context

The single most effective, lowest-cost defense in industrial cybersecurity is network segmentation — separating office IT from OT so a breach of one doesn't automatically become a breach of the other.

### Technical approach

The design follows the Purdue Enterprise Reference Architecture and applies IEC 62443 to define which systems may communicate. This wasn't left as a diagram — it was rebuilt as a working network in Cisco Packet Tracer, with a router actively enforcing the boundary.

### Findings

Testing confirmed the enforcement works: a direct connection attempt from office to control network failed, while traffic through the designated buffer zone routed correctly.

### Why it matters

A design that's built and tested demonstrates the control actually works. Every other project below builds on this network directly.

*Full evidence: [github.com/carmelin-neto/ot-purdue-model-mapping](https://github.com/carmelin-neto/ot-purdue-model-mapping)*

---

## Project 2: Why Standard IT Security Tools Are Dangerous in OT

### Business context

A common, costly mistake is applying ordinary IT security practices — like routine vulnerability scanning — to OT environments without modification.

### Technical approach

Industrial controllers run purpose-built operating systems not designed to withstand a flood of unexpected traffic. Documented research shows ordinary scanning tools have crashed live controllers during routine testing.

### Findings

The safer, industry-standard alternative is passive monitoring — observing traffic without sending anything to the device directly.

### Why it matters

This judgment separates competent OT practice from IT practice applied carelessly to the wrong environment, and is put into direct practice in Project 5.

*Full evidence: [github.com/carmelin-neto/ot-active-scanning-risk](https://github.com/carmelin-neto/ot-active-scanning-risk)*

---

# OT/ICS Security Portfolio — Project Case Studies

A technical summary for non-technical readers

## Project 3: Attack Testing and Building a Detection Pipeline

### Business context

This project moves from designing defenses to actively testing them, then builds detection to catch an attack if one succeeds — mirroring a real security team's red team / blue team workflow.

### Technical approach

A direct data manipulation attempt against the Modbus protocol (which has no built-in authentication) — the write command was accepted, but the real sensor reading never changed, because read-only and writable data are kept in genuinely separate memory spaces. An interception attempt (ARP spoofing) failed, because the target device ignores unsolicited network announcements by design.

### Findings

A custom Python detection script, a custom Suricata IDS rule, and a scheduled Splunk alert were built and validated to correctly fire on real attack traffic while staying silent on legitimate traffic.

### Why it matters

This becomes the shared evidence base for several projects below: the incident this system responds to, the risk it's scored against, the vulnerability it stems from, the access restriction that blocks it a second way, and the AI triage layer built on top of it.

*Full evidence: [github.com/carmelin-neto/ics-modbus-attack-testing](https://github.com/carmelin-neto/ics-modbus-attack-testing)*

---

## Project 4: Responding to the Attack — Incident Response Walkthrough

### Business context

Detecting an attack is only half the job. A team needs a disciplined process for what happens next — one that accounts for physical safety, not just data security.

### Technical approach

Using the exact attack and detection tooling from Project 3, this works through a complete six-phase incident response. The key decision: the obvious IT instinct — disconnect immediately — is deliberately rejected, since abruptly cutting a live industrial process can leave equipment in an unsafe state. A targeted containment method is used instead.

### Findings

The full response was walked through with real timestamps and reasoning, including an honest review of what a real deployment would still need.

### Why it matters

This shows judgment under a realistic operational constraint, and the self-awareness to name a process gap rather than hide it.

*Full evidence: [github.com/carmelin-neto/ot-incident-response-scenario](https://github.com/carmelin-neto/ot-incident-response-scenario)*

---

## Project 5: Finding What's Exposed — Vulnerability Assessment

### Business context

Project 2 argued that careless scanning is dangerous in OT. This project puts that judgment into practice.

### Technical approach

The OT environment was scanned deliberately slowly and narrowly — including honestly documenting the scan's own blind spot. A separate, intentionally vulnerable general-IT system was then scanned at full aggressiveness, since that target was built to withstand it.

### Findings

Five real, verified vulnerabilities were found with severity ratings and fixes — including two that turned out to be historical supply-chain compromises rather than ordinary coding bugs.

### Why it matters

Matching scan intensity to target sensitivity is exactly the judgment call a real assessment requires.

*Full evidence: [github.com/carmelin-neto/ot-vulnerability-assessment](https://github.com/carmelin-neto/ot-vulnerability-assessment)*

---

# OT/ICS Security Portfolio — Project Case Studies

A technical summary for non-technical readers

## Project 6: Turning Findings Into Priorities — Risk Assessment

### Business context

A list of vulnerabilities doesn't tell a business what to fix first. This project scores every finding from Projects 3 and 5 using IEC 62443, turning technical findings into a prioritized business decision.

### Technical approach

Each risk was scored on likelihood and impact, starting at the single most critical device and widening to the network as a whole — showing numerically how much each safeguard actually reduces risk.

### Findings

One risk was left honestly unresolved and flagged as the portfolio's single open action item — because an assessment where everything is conveniently already fixed isn't credible.

### Why it matters

This turns "I found some problems" into "here is what to fix first, and why."

*Full evidence: [github.com/carmelin-neto/ot-risk-assessment](https://github.com/carmelin-neto/ot-risk-assessment)*

---

## Project 7: Restricting Who Can Do What — Access Control Design

### Business context

Network segmentation controls which devices can reach which. It says nothing about what a person can do once they've legitimately reached one. This project adds that layer.

### Technical approach

Four roles were defined — operator, engineer, security analyst, and maintenance vendor — each given only the access their job requires. The security analyst role has no direct control-equipment access at all; vendor access is restricted to a single device and set to expire automatically.

### Findings

The same attack from Project 3 is blocked a second, independent way — through role restrictions, not just network rules.

### Why it matters

This is a design document, not yet a deployed and tested system — the lab environment doesn't support the multi-user login needed to enforce it live. That limitation is stated directly, consistent with the rest of this portfolio.

*Full evidence: [github.com/carmelin-neto/ot-access-control-design](https://github.com/carmelin-neto/ot-access-control-design)*

---

## Project 8: Protecting a Web Application — WAF Deployment and a Real Cloud Limitation

### Business context

Most business systems today aren't industrial controllers — they're web applications. This project closes that gap, testing a web application firewall the same way the rest of this portfolio tests everything else.

### Technical approach

A real, deliberately vulnerable web application was deployed, then a firewall (ModSecurity) was placed in front of it and tested in three stages: no protection, detect-but-don't-block, and full blocking — against a realistic SQL injection attack.

### Findings

Without protection, the attack succeeded and leaked internal database details. Detecting-but-not-blocking correctly logged and flagged the attack. Blocking mode stopped it outright, and a final check confirmed legitimate traffic still passed normally. A cloud-firewall extension hit a genuine limitation — it requires owning a registered domain, even on a free tier — documented honestly rather than papered over.

### Why it matters

This shows the same staged verification approach applied to a different technology, and the professional judgment to document a real infrastructure obstacle honestly.

*Full evidence: [github.com/carmelin-neto/waf-deployment-lab](https://github.com/carmelin-neto/waf-deployment-lab)*

---

# OT/ICS Security Portfolio — Project Case Studies

A technical summary for non-technical readers

## Project 9: A Home Security Operations Center with AI-Assisted Alert Triage

### Business context

Security teams increasingly work alongside AI tools that help sort through alerts — but an AI tool is only useful if someone knows how to check its work.

### Technical approach

A security monitoring platform (Wazuh) was deployed and connected to the detection system from Project 3. A locally-run AI model was given a real detected alert and asked to classify its severity, summarize what happened, and recommend a next step.

### Findings

Loading historical alert data surfaced a genuine technical obstacle, solved and documented. The AI's triage was checked line by line against the real alert data: it got the overall situation right but made two specific, real mistakes — citing the wrong technical detail and missing an important distinction the raw data showed. Both are documented plainly.

### Why it matters

The value isn't that the AI performed perfectly — it didn't. The value is proving the ability to use an AI tool critically: real benefit, with errors caught rather than accepted blindly. That's a materially different and more useful claim than simply "I used an AI tool."

*Full evidence: [github.com/carmelin-neto/home-soc-ai-triage](https://github.com/carmelin-neto/home-soc-ai-triage)*

---

## Closing Note

Every technical claim in this document is backed by a screenshot, a log file, or tested code in the linked repositories — nothing here is a description of intended work; it's a summary of work that was actually built, run, and verified. Where something wasn't fully tested or resolved, that's stated plainly rather than hidden. That distinction is the entire point of this portfolio.